# Introduction to Blockchain and Smart Contracts

## Lecture 12: Tokens, NFTs & Exchanges

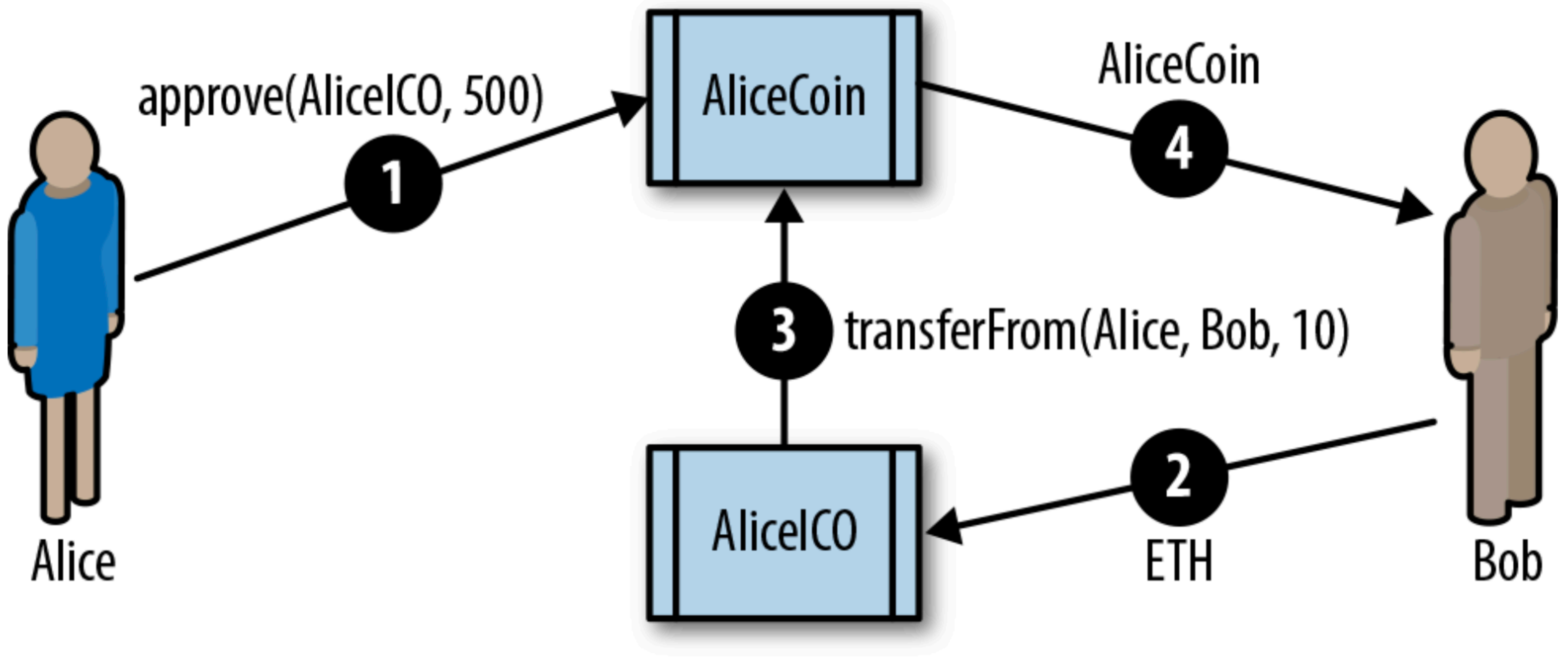
Subodh Sharma | IIT Delhi | March 14, 2026

# ERC20 Token

- ERC20 compliant token must at least provide following functions and events:
    - totalSupply
    - balanceOf(addr)
    - transfer
    - transferFrom
    - approve
    - allowance
    - EVENTS: Approval, …

```
contract ERC20 {
   function totalSupply() constant returns (uint theTotalSupply);
   function balanceOf(address _owner) constant returns (uint balance);
   function transfer(address _to, uint _value) returns (bool success);
   function transferFrom(address _from, address _to, uint _value) returns
      (bool success);
   function approve(address _spender, uint _value) returns (bool success);
   function allowance(address _owner, address _spender) constant returns
      (uint remaining);
   event Transfer(address indexed _from, address indexed _to, uint _value);
   event Approval(address indexed _owner, address indexed _spender, uint _value);
}
```

# ERC20 Token - ICO




- Two txns are made:
  - Approve and transferFrom
- ERC20 Token implementations
  - Consensys EIP 20
  - OpenZeppelin
  - Uniswap

# ERC20 Token - Issues

- Subtle differences on how tokens and ether are handled
  - Ether is sent using the send function and accepted by a *payable function*
    - Tokens are sent using `transfer/approve and transferFrom` and **don’t trigger state change of the recepient’s address**
    - `token.transfer(address(vault), 100):` Anita sends 100 tokens to a contract address —> balance of Anita goes down —> balance of vault goes down —> a Transfer event is emitted
      - No automatic callback to the vault contract

# ERC20 Token - Issues

- Subtle differences on how tokens and ether are handled
  - Sending of ether incurs gas. To send tokens you also need ether. You cannot pay for transaction's gas with tokens and the token contract can't pay the gas for you.
  - `usdc.transfer(bob, 100e6)` —> Anita submits an Ethereum txn that calls USDC contract to transfer amount to Bob —> Txn gas is to be paid in ETH
- Upgraded ERC20 protocol — **ERC777**
  - **ERC777:**
    - adds hooks; for instance receiver callback can be supported
    - Recipients can initiate a receiving of tokens (through hooks) when sender has no withdrawal logic available
    - **ERC20 has a race condition** (leading to front-running attacks) during changing of allowance from one value to another

# ERC721 - (Non Fungible Tokens)

- The ERC20 token doesn’t track the provenance of tokens
- This is a standard for the “deed”
- NFTs track ownership of a unique thing
  - Which means each token as a unique identifier
  - Ownership is tracked per token
  - Transfer is accomplished per token
- ERC721 doesn’t place any limitation on the nature of the thing
-

# ERC721 - (Non Fungible Tokens)

- Internal data structure: `mapping (uint256 => address) private DeedOwner`
    - In ERC20: `balance[Anita] = 100`
    - In ERC721: DeedOwner[101] = Anita
- Main functions of ERC721:
    - `balanceOf(address owner)`: returns how many NFTs an address owns
    - `ownerOf(uint256 tokenId)`
    - `transferFrom(address From, address To, unit256 tokenId)`
    - `safeTransferFrom(…)`: transfer fails of the recipient doesn't have the logic to receive NFTs
    - `approve(address To, uint256 tokeId)`

# ERC721 - (Non Fungible Tokens)

- NFT Metadata: How would the NFT token display name, image, description
  - name() —> collection name
  - symbol() —> collection symbol
  - tokenURI(tokenId) —> URI pointing to metadata JSON
- Usecases: Tickets, Certificates, Identity-linked proofs, real-world assets
- Why not just use an immutable Database:
  - Shared public verification, decentralisation
- Weakness of ERC721: the token doesn’t contain the asset; off chain storage risk; high gas costs in minting and burning NFTs one by one, privacy-preservation?

# Other ERC Standards worth learning

- **ERC1155** — Multi-token standard — single contract to represent many token types, including fungible, non-fungible, and semi-fungible assets.
  - Motivations: Cheaper batch operations; simpler asset management;
- **ERC4626 -** Standardises tokenised vaults — a vault accepts an ERC20 asset and issues shares representing a claim on that vault

# Exchanges

# Crypto Exchanges

- Important functions:
  - **Price discovery**: rate at which one currency can be exchanged for another
  - **Matching:** Connecting buyers and sellers
  - **Custody/Settlement**
  - **Liquidity provisions**

# Crypto Exchanges

- Types of exchanges —
  - **Decentralised**: Smart contract based trading system. Wallets are self-custodial.
    - Eg: Uniswap
    - Two broad categories:
      - Orders can be posted either off-chain or on-chain using **order books** but settlement is always on-chain
      - Users trade against **liquidity pools** rather than traditional order books (**A**utomated **M**arket **M**aker)
        - No counterparty; price rate algorithmically fixed; Liquidity providers — anyone can put their money into the pool to earn a share of the trading fee.

# Decentralised Exchange

- Example: Anita wants to swap ETH for USDC on a DEX
  - Anita opens a wallet such as **MetaMask**.
  - She connects it to a DEX interface.
  - She selects “swap 1 ETH for USDC.”
  - The DEX contract quotes an output based on current pool state.
  - Anita signs a transaction with her wallet.
  - The blockchain executes the swap.
  - Anita receives USDC directly into her wallet.

# Crypto Exchanges: Centralised

- Types of exchanges —
  - **Centralised**: Binance, Coinbase, …
    - Wallets under exchange control, internal ledger, user accounts, ..
    - Custodian of wallets and deposited coins
    - Easier to regulate

# Proof Requirements for CEXs

- Several security scares:
  - Bank run: When simultaneously many people want to withdraw
  - Bank owners are crooks
  - Insider attack ..
- Proofs of
  - Assets, Reserves, Liabilities, Solvency